

Product Requirements Document: Desktop Keyboard Assistant

1. Product Overview

Product Name

Working title: **Keyboard Assistant**

Product Summary

Keyboard Assistant is a privacy-first desktop typing assistant that provides real-time typo correction, capitalisation correction, apostrophe correction, smart word suggestions, and next-word prediction across supported desktop applications.

The product should feel similar to the intelligence of an iOS keyboard, but for desktop typing. It should work immediately after installation using built-in correction rules, dictionaries, and default suggestion logic. Over time, it should become more accurate by learning locally from the user's typing habits, accepted suggestions, ignored suggestions, custom vocabulary, and app-specific behaviour.

The app should remain minimal from the user's perspective. It should not require users to manually switch between different modes. Instead, it should automatically adjust its behaviour based on focused application, text field type, confidence score, and user history.

Product Vision

To build a lightweight, private, intelligent desktop keyboard assistant that helps users type faster and more accurately across their computer, without sending their typing data to the cloud.

Core Product Promise

The app should feel like iOS keyboard intelligence for desktop: fast, subtle, useful, private, and adaptive.

2. Goals and Non-Goals

Goals

1. Provide real-time spelling and typo correction across supported desktop apps.
2. Correct common capitalisation mistakes automatically when confidence is high.
3. Add or correct apostrophes in common contractions.
4. Show a small interactive suggestion box near the cursor.
5. Allow users to accept, ignore, or choose alternative suggestions quickly using keyboard shortcuts.

6. Predict likely next words or phrases based on sentence context and user habits.
7. Learn locally from user behaviour over time.
8. Store all user data locally by default.
9. Work immediately after fresh installation without requiring training.
10. Automatically reduce or disable assistance in unsuitable contexts such as code editors, terminals, password fields, games, and browser search bars.
11. Provide a minimal settings frontend for controlling behaviour, privacy, appearance, and app exclusions.
12. Use a modular local AI layer that can support different local models later.
13. Remain usable on most modern computers through adaptive performance behaviour.

Non-Goals for Initial Version

1. No cloud-based correction or cloud-based AI processing.
 2. No mandatory account system.
 3. No LangChain dependency for the real-time correction pipeline.
 4. No manual user-facing writing modes such as "Email Mode", "Coding Mode", or "Rewrite Mode".
 5. No full custom keyboard replacement at the operating-system level.
 6. No mobile app support in the initial version.
 7. No multiplayer/team dictionary features in the initial version.
 8. No true LLM fine-tuning in the first version.
 9. No guaranteed support for every desktop application on day one.
-

3. Target Users

Primary Users

1. Students writing essays, reports, messages, emails, and notes.
2. Professionals who type frequently across email, messaging, documents, and browser-based tools.
3. Users who want better typing assistance than the default desktop spellchecker.
4. Privacy-conscious users who do not want their typed content sent to cloud services.
5. Fast typists who want correction and suggestions without slowing down.

Secondary Users

1. Multilingual users who may later benefit from mixed-language correction.
 2. Developers who want assistance outside code editors but do not want corrections interfering with code.
 3. Users with repeated spelling or grammar habits who want a tool that adapts to them.
-

4. Core Principles

4.1 Minimal User Experience

The app should stay out of the way. It should not require users to constantly configure modes or manually trigger basic corrections.

4.2 Local-First Privacy

Typed data, correction history, personal dictionary, and learned habits should be stored locally by default. No cloud processing should be required for core functionality.

4.3 Fast Before Smart

The app should prioritise low-latency correction before advanced AI suggestions. Basic corrections should happen instantly without relying on a local LLM.

4.4 Adaptive Intelligence

The app should work from day one using built-in knowledge, then improve over time through local learning.

4.5 Conservative Auto-Correction

The assistant should only auto-correct when confidence is high. For uncertain cases, it should show suggestions instead of changing the user's text automatically.

4.6 Context Awareness Without Visible Modes

The app should automatically adapt to the focused application and text field type. Users should not need to manually select a mode before typing.

5. User Stories

Basic Typo Correction

As a user, I want obvious typos to be corrected automatically so that I can type quickly without stopping to fix small mistakes.

Example:

```
teh → the  
recieve → receive  
definatly → definitely
```

Apostrophe Correction

As a user, I want common contractions to be fixed automatically so that my writing looks correct.

Example:

```
dont → don't  
cant → can't  
im → I'm  
ill → I'll
```

Capitalisation Correction

As a user, I want the assistant to capitalise words when appropriate.

Example:

```
i am going home → I am going home  
hello. how are you → Hello. How are you
```

Suggestion Acceptance

As a user, I want to accept the main suggestion quickly using the keyboard, without touching the mouse.

Expected behaviour:

```
Space / Tab = accept main suggestion depending on context  
Esc = ignore suggestion  
Alt + 1 / 2 / 3 = choose alternative suggestions
```

Ignore Suggestion

As a user, I want to ignore a suggestion and continue typing naturally.

Personal Learning

As a user, I want the app to learn the words, phrases, names, and typing style I commonly use so that suggestions become more accurate over time.

Fresh Install Usefulness

As a new user, I want the app to be useful immediately after installation, before it has learned anything about me.

App-Specific Behaviour

As a user, I want the app to behave differently depending on where I am typing, without me manually changing settings.

Example:

```
Email: active correction and suggestions
Messaging: casual correction and suggestions
Browser search bar: minimal assistance
Code editor: limited or disabled
Terminal: disabled
Password field: disabled
```

Privacy Control

As a user, I want to see and control what the app stores locally, and I want to be able to delete all learned data.

6. Functional Requirements

6.1 Global Typing Capture

The app must be able to monitor typed input across supported desktop applications.

Requirements

1. Capture keyboard input in real time.
2. Detect the currently focused application.
3. Detect or estimate the focused text field when possible.
4. Track recent text around the cursor when available.
5. Avoid capturing text in sensitive fields such as password inputs.
6. Pause or disable itself in unsupported or sensitive environments.
7. Handle fast typing without dropping input.
8. Avoid interfering with normal keyboard shortcuts.

Notes

System-level input handling is one of the most technically difficult parts of the product. The implementation should support fallback behaviour when full text context is unavailable.

6.2 Text Context Extraction

The app should understand enough nearby text to make useful suggestions.

Requirements

1. Extract the current word being typed.
2. Extract the previous few words when possible.
3. Extract the current sentence when possible.
4. Detect cursor location and update suggestions based on cursor position.
5. Handle edited text when the cursor moves backward into an earlier sentence.
6. Cancel outdated suggestions when the user continues typing.

Fallback Behaviour

If the app cannot read surrounding text from the target application, it should rely on recently typed keystrokes only.

6.3 Correction Engine

The correction engine should be the core of the app. It should not rely entirely on an LLM.

Required Correction Types

1. Common typo correction.
2. Keyboard-neighbour typo correction.
3. Missing apostrophe correction.
4. Capitalisation correction.
5. Repeated character correction.
6. Basic spacing correction.
7. Contextual word correction when confidence is high.

Examples

```
teh → the
dont → don't
i → I
helllo → hello
this is → this is
I will meat you later → I will meet you later
```

Confidence-Based Behaviour

The system should assign a confidence score to each correction.

Confidence Level	Behaviour
Very high	Auto-correct
Medium	Show suggestion
Low	Do nothing
User-specific high	Auto-correct if user has repeatedly accepted similar corrections
User-specific low	Stop suggesting if user repeatedly ignores correction

6.4 Suggestion Engine

The suggestion engine should generate and rank possible corrections, word completions, and next-word predictions.

Requirements

1. Generate up to three suggestions at a time.
2. Rank suggestions by confidence, context, user history, and word frequency.
3. Show the highest-confidence suggestion as the primary suggestion.
4. Provide alternative suggestions when useful.
5. Avoid showing suggestions when confidence is too low.
6. Avoid distracting the user with constant unnecessary suggestions.
7. Update the suggestion box smoothly as the user types.

Suggestion Types

1. Current-word correction.
2. Current-word completion.
3. Next-word prediction.
4. Short phrase prediction.
5. Apostrophe correction.
6. Capitalisation correction.
7. Context-aware replacement.

6.5 User Interaction

The user should be able to interact with suggestions without breaking typing flow.

Keyboard Controls

Action	Shortcut
Accept main suggestion	Space or Tab, depending on context
Ignore suggestion	Esc
Choose first suggestion	Alt + 1
Choose second suggestion	Alt + 2
Choose third suggestion	Alt + 3
Revert last correction	Backspace immediately after correction or Ctrl + Z
Pause assistant quickly	Tray menu or configurable shortcut

Requirements

1. User must be able to ignore suggestions naturally.
2. User must be able to accept suggestions without using the mouse.
3. User must be able to undo corrections quickly.
4. The app should not hijack normal typing behaviour.
5. Shortcuts should be configurable later.

6.6 Suggestion Box UI

The suggestion box should be small, polished, and non-intrusive.

Requirements

1. Show near the current cursor position.
2. Follow the cursor as the user types or moves the caret.
3. Avoid covering the typed text when possible.
4. Animate smoothly when suggestions change.
5. Show the main suggestion clearly.
6. Optionally show up to two alternative suggestions.
7. Support light and dark themes.
8. Support adjustable size and opacity.
9. Automatically hide when no useful suggestion exists.
10. Automatically hide in disabled or sensitive contexts.

Example UI Content

don't do not don't forget

6.7 Local Learning System

The app should become smarter through local adaptive learning.

Requirements

The app should learn from:

1. Accepted suggestions.
2. Ignored suggestions.
3. Reverted corrections.
4. Words repeatedly typed by the user.
5. Custom words added by the user.
6. Frequently used phrases.
7. Application-specific typing habits.
8. User-specific capitalisation patterns.
9. User-specific contraction preferences.

What the App Should Learn

1. Personal vocabulary.
2. Names and proper nouns.
3. Common phrases.
4. Preferred spelling variants.
5. Suggestions the user dislikes.
6. Apps where the user wants less assistance.
7. Writing habits that improve prediction quality.

Important Requirement

The initial version should use adaptive local learning, not full LLM fine-tuning.

The product should describe this as:

The app continuously adapts to the user's typing behaviour through local learning, including accepted suggestions, ignored suggestions, custom vocabulary, phrase frequency, and app-specific typing patterns.

6.8 Fresh Install Intelligence

The app must be useful immediately after installation.

Built-In Data and Logic

The app should include:

1. Default English dictionary.
2. Common typo map.
3. Common contraction rules.
4. Capitalisation rules.
5. Basic grammar rules.
6. Common phrase prediction data.
7. Word frequency list.
8. Default correction confidence scoring.
9. Default sensitive app and field rules.
10. Default app exclusion list.

Requirement

The user should not need to train the app before it becomes useful.

6.9 Personal Dictionary

The app should maintain a local personal dictionary.

Requirements

1. Add custom words manually.
2. Add custom words automatically after repeated usage.
3. Mark words as "never correct".
4. Remove words from the dictionary.
5. Import/export dictionary later.
6. Store dictionary locally.

Examples

```
Exeter  
Qwen  
Ollama  
Aeroad  
R8070  
Yew
```

6.10 App Awareness and Automatic Behaviour

The assistant should automatically adapt based on the focused app.

Default Behaviour Examples

Context	Behaviour
Email client	Full assistance
Messaging app	Active but casual assistance
Browser text area	Active assistance
Browser search bar	Minimal assistance
Code editor	Limited or disabled
Terminal	Disabled
Password field	Disabled
Game	Disabled
Remote desktop	Disabled by default

Requirements

1. Detect active application.
2. Apply default app rules.
3. Allow users to exclude apps manually.
4. Allow users to re-enable assistance for excluded apps.
5. Learn app-specific preferences over time.
6. Disable in sensitive fields.
7. Avoid interfering with code, command line input, and games.

Minimal User-Facing Status

Instead of showing complex modes, the app should display simple statuses:

```
Assistant On
Assistant Limited
Assistant Off
```

6.11 Local AI Model Layer

The app should support a local AI model layer, but the product must not depend entirely on it.

Requirements

1. Local AI should enhance suggestions, not replace the core correction engine.
2. The app should remain functional without a local model.
3. The model provider should be modular and replaceable.
4. The initial implementation may support local providers such as Ollama or other local inference backends.
5. The system should support small, efficient models for common machines.
6. The local model should only be called when useful, not on every keystroke.
7. Outdated AI suggestions should be cancelled when the user continues typing.
8. AI calls should never block typing.

AI Use Cases

1. Context-aware word suggestions.
2. Next-word prediction.
3. Short phrase prediction.
4. Sentence-level grammar suggestions.
5. Meaning-aware typo correction.
6. Optional future sentence refinement.

Important Constraint

No cloud AI service should be required for the initial product.

6.12 Local Database

The app should use a local database for user data and learning data.

Recommended Database

SQLite for the initial version.

Possible Tables

```
settings
app_profiles
personal_dictionary
correction_history
accepted_suggestions
ignored_suggestions
reverted_corrections
phrase_frequency
word_frequency_user
```

```
model_settings
privacy_settings
```

Requirements

1. Store user learning data locally.
2. Support clearing all data.
3. Support resetting learning data without deleting app settings.
4. Support exporting/importing user data later.
5. Avoid storing sensitive fields.
6. Avoid storing password-like input.

6.13 Settings Frontend

The settings UI should be simple and minimal.

Main Settings Sections

1. General
2. Apps
3. Dictionary
4. Learning
5. Appearance
6. Privacy
7. Local AI

General Settings

1. Assistant on/off.
2. Start on system startup.
3. Correction strength.
4. Suggestion shortcuts.
5. Pause assistant.

Correction Strength

The user should be able to choose:

```
Light
Balanced
Aggressive
```

Default: Balanced.

Setting	Behaviour
Light	Fewer auto-corrections, more suggestions
Balanced	Auto-correct obvious mistakes only
Aggressive	More proactive corrections and suggestions

Apps Settings

1. View detected apps.
2. Enable/disable assistant per app.
3. Reset app-specific behaviour.
4. Add app to exclusion list.

Dictionary Settings

1. View personal dictionary.
2. Add word.
3. Remove word.
4. Mark word as never correct.

Learning Settings

1. Enable/disable local learning.
2. Clear learned data.
3. Reset accepted/ignored suggestion history.
4. Reset phrase learning.

Appearance Settings

1. Theme: system, light, dark.
2. Suggestion box size.
3. Suggestion box opacity.
4. Animation on/off.

Privacy Settings

1. View local data summary.
2. Delete all local data.
3. Disable learning.
4. Manage sensitive app exclusions.
5. Privacy explanation.

Local AI Settings

1. Enable/disable local AI suggestions.
2. Select local model provider.
3. Show model status.
4. Test model response.

5. Configure model path or backend later.
-

7. Privacy and Security Requirements

Privacy Requirements

1. All user typing data should remain local by default.
2. No typed content should be sent to cloud services in the initial version.
3. The app should clearly explain what data is stored.
4. The user should be able to delete all stored data.
5. The user should be able to disable learning.
6. The app should not store text from password fields.
7. The app should not store sensitive fields when detectable.
8. The app should provide app-level exclusions.

Sensitive Contexts

The assistant should be disabled by default in:

```
Password fields
Banking websites/apps where detectable
Password managers
Terminal apps
Code editors
Games
Remote desktop apps
Private/incognito browser windows where detectable
```

Security Requirements

1. Local database should be stored securely.
 2. Sensitive data should not be logged.
 3. Debug logs should avoid storing raw typed text.
 4. The app should not expose a network API by default.
 5. Any local AI backend communication should remain local.
-

8. Performance Requirements

Typing Responsiveness

1. The app must not slow down typing.
2. Basic correction should happen instantly.
3. Suggestion updates should feel real-time.

4. The app should support fast typists above 120 WPM.
5. AI suggestions should run asynchronously.
6. AI suggestions should be cancellable if outdated.
7. The app should degrade gracefully on lower-end machines.

Performance Strategy

1. Use rule-based and dictionary-based correction first.
2. Use cached suggestions where possible.
3. Avoid calling the local AI model on every keystroke.
4. Trigger local AI only after pauses, sentence boundaries, or low-confidence cases.
5. Keep keyboard capture separate from AI processing.
6. Never block the user's typing while waiting for AI output.

Adaptive Behaviour

The app should automatically adjust based on system performance.

Example:

System Capability	Behaviour
Low performance	Rules, dictionary, lightweight suggestions
Average performance	Rules, dictionary, local learning, limited local AI
High performance	More frequent local AI suggestions
No local model	Core correction still works

9. Technical Architecture

High-Level Components

```
Keyboard/Input Listener
    ↓
Focused App Detector
    ↓
Text Context Manager
    ↓
Sensitive Context Filter
    ↓
Correction Engine
    ↓
Suggestion Generator
    ↓
```



```
Suggestion Ranker
  ↓
Suggestion Overlay UI
  ↓
User Feedback Collector
  ↓
Local Learning Engine
  ↓
Local Database
```

Additional Components

```
Settings Frontend
Local AI Service Layer
Personal Dictionary Manager
App Profile Manager
Privacy Manager
Performance Manager
Logging and Diagnostics System
```

10. Suggested Technology Stack

Core Language

Python for the main logic and early development.

Database

SQLite for local storage.

Local AI

A modular local AI provider layer.

Possible backends:

```
Ollama
llama.cpp
ONNX Runtime
Other local inference backends later
```

Frontend

A lightweight desktop frontend for settings.

Possible options:

```
PySide6 / Qt
Tauri with Python backend
Electron with Python backend
Custom native frontend later
```

Overlay UI

The suggestion box should be implemented as a lightweight always-on-top overlay window that can follow the text cursor when possible.

Packaging

The app should eventually be packaged as a normal desktop installer.

Target initial OS should be selected before implementation. Windows is likely the first target because desktop keyboard hooks and user environment are currently Windows-focused.

11. Data Model Draft

settings

Stores general app settings.

```
id
key
value
updated_at
```

app_profiles

Stores behaviour per application.

```
id
app_name
app_identifier
assistant_status
```

```
correction_strength
learning_enabled
created_at
updated_at
```

personal_dictionary

Stores user vocabulary.

```
id
word
source
frequency
never_correct
created_at
updated_at
```

correction_history

Stores correction events.

```
id
original_text
corrected_text
correction_type
confidence
app_identifier
accepted
reverted
created_at
```

accepted_suggestions

Stores suggestions accepted by the user.

```
id
input_text
suggestion_text
suggestion_type
app_identifier
created_at
```

ignored_suggestions

Stores suggestions ignored by the user.

```
id
input_text
suggestion_text
suggestion_type
app_identifier
created_at
```

phrase_frequency

Stores common local phrases.

```
id
phrase
frequency
app_identifier
last_used_at
```

word_frequency_user

Stores user-specific word frequency.

```
id
word
frequency
app_identifier
last_used_at
```

12. MVP Scope

MVP Must Have

1. Global keyboard listener.
2. Focused app detection.
3. Basic context tracking from typed characters.
4. Common typo correction.
5. Apostrophe correction.
6. Basic capitalisation correction.
7. Suggestion box near cursor or near active text area.

8. Accept suggestion with keyboard.
9. Ignore suggestion with Esc.
10. Revert last correction.
11. Local SQLite database.
12. Personal dictionary.
13. Correction history.
14. Local learning from accepted and ignored suggestions.
15. App exclusion list.
16. Disable in password fields where detectable.
17. Minimal settings UI.
18. Local-only data storage.
19. Basic local AI integration layer, even if optional.
20. App remains usable without local AI.

MVP Should Have

1. Top 3 suggestions.
2. App-specific learning.
3. Phrase frequency learning.
4. Smooth suggestion box animation.
5. Correction strength setting.
6. Light/dark appearance.
7. Privacy dashboard.
8. Local model status page.

MVP Could Have

1. Better next-word prediction.
2. Context-aware grammar suggestions.
3. Import/export dictionary.
4. Configurable shortcuts.
5. More advanced cursor-following.
6. Selected text rewriting.

Not in MVP

1. Cloud sync.
 2. Accounts.
 3. Team dictionaries.
 4. Full multilingual support.
 5. Mobile support.
 6. True model fine-tuning.
 7. Browser extension.
 8. Full support for every desktop app.
-

13. Acceptance Criteria

Typo Correction

- Given the user types a common typo such as `teh`, the app should suggest or correct it to `the`.
- Given the correction confidence is high, the app may auto-correct.
- Given confidence is medium, the app should show a suggestion instead of auto-correcting.

Apostrophe Correction

- Given the user types `dont`, the app should suggest or correct to `don't`.
- Given the user repeatedly rejects a contraction correction, the app should reduce confidence for that suggestion.

Capitalisation

- Given the user types `i am`, the app should correct or suggest `I am`.
- Given the user starts a new sentence without capitalisation, the app should suggest or correct capitalisation.

Suggestion UI

- The suggestion box should appear near the cursor when there is a useful suggestion.
- The suggestion box should hide when there is no useful suggestion.
- The suggestion box should update as the user types.
- The suggestion box should not block typing.

User Control

- The user should be able to accept a suggestion using the keyboard.
- The user should be able to ignore a suggestion using Esc.
- The user should be able to revert the last correction.

Privacy

- The app should store learning data locally.
- The user should be able to clear all local learning data.
- The app should not require internet access for core correction.
- The app should not capture or store password field content where detectable.

Local Learning

- If the user repeatedly accepts a suggestion, its future ranking should increase.
- If the user repeatedly ignores a suggestion, its future ranking should decrease.
- If the user repeatedly types a custom word, the app should learn it or suggest adding it to the personal dictionary.

App Awareness

- The app should be disabled in excluded apps.
 - The app should be disabled in terminal apps by default.
 - The app should be limited or disabled in code editors by default.
 - The user should be able to change app-specific behaviour.
-

14. Edge Cases

1. User types very quickly.
 2. User moves cursor into the middle of a sentence.
 3. User deletes text after a suggestion appears.
 4. User accepts a suggestion, then immediately presses backspace.
 5. User types in a password field.
 6. User types in a code editor.
 7. User types in a terminal.
 8. User uses unusual names or slang.
 9. User types mixed languages.
 10. User types in a browser search bar.
 11. User switches apps while a suggestion is visible.
 12. User uses keyboard shortcuts while the assistant is active.
 13. Local AI model is unavailable.
 14. Local AI model is slow.
 15. Cursor location cannot be detected.
 16. Focused application does not expose text context.
 17. User disables learning.
 18. Database becomes corrupted.
 19. App crashes during typing.
 20. Suggestion becomes outdated before it is shown.
-

15. Risks and Mitigations

Risk: System-Level Text Handling Is Difficult

Different desktop apps expose text fields differently.

Mitigation:

1. Start with Windows support first.
2. Use fallback typed-buffer context when full text extraction is unavailable.
3. Maintain app-specific compatibility rules.
4. Avoid promising full support for every app in MVP.

Risk: Local AI Is Too Slow

Local models may not respond quickly enough for real-time typing.

Mitigation:

1. Do not call AI on every keystroke.
2. Use rules and dictionary correction first.
3. Run AI asynchronously.
4. Cancel outdated AI requests.
5. Keep the app useful without AI.

Risk: Bad Corrections Annoy Users

Aggressive correction may damage trust.

Mitigation:

1. Use conservative auto-correction.
2. Require high confidence before changing text automatically.
3. Allow immediate undo.
4. Learn from ignored and reverted suggestions.

Risk: Privacy Concerns

Users may worry about a typing assistant monitoring input.

Mitigation:

1. Make local-only storage clear.
2. Disable sensitive fields.
3. Provide delete-all-data controls.
4. Avoid cloud processing.
5. Avoid storing raw sensitive text in logs.

Risk: Suggestion Box Feels Distracting

A bad overlay could annoy users.

Mitigation:

1. Keep the UI small.
2. Hide when confidence is low.
3. Allow opacity and size adjustment.
4. Use smooth but subtle animation.

16. Future Expansion

Potential future features:

1. Better local AI models.
 2. Optional selected-text rewriting.
 3. Import writing style from documents.
 4. Multilingual support.
 5. Cantonese/English mixed typing support.
 6. Cloud sync as an optional privacy-controlled feature.
 7. Team dictionary support.
 8. Browser extension for improved web text-field integration.
 9. Advanced grammar checking.
 10. Voice dictation cleanup.
 11. User-defined text shortcuts.
 12. Academic writing suggestions.
 13. Email reply suggestions.
 14. More advanced personalisation.
 15. Optional encrypted backup.
-

17. Open Questions

1. Which operating system should be the first official target?
 2. Should the first version be Windows-only?
 3. Should suggestions be accepted with Space, Tab, or both depending on context?
 4. How aggressive should default auto-correction be?
 5. Which apps should be disabled by default?
 6. Should the app store full correction history or only anonymised correction patterns?
 7. Should local AI be bundled, optional, or user-installed separately?
 8. Which local AI backend should be supported first?
 9. Should the overlay be built with Qt, Tauri, Electron, or another UI stack?
 10. How should the app detect browser search bars versus normal text areas?
 11. How should the app handle code editors: fully disabled or comments/strings only?
 12. Should the app support only English in MVP?
 13. Should the app have an onboarding tutorial?
 14. Should users be asked for permission before local learning starts?
-

18. Recommended Initial Build Plan

Phase 1: Foundation

1. Create project structure.
2. Implement global keyboard listener.
3. Implement focused app detector.

4. Implement local SQLite database.
5. Implement typed-buffer context tracking.
6. Implement common typo correction rules.
7. Implement apostrophe and capitalisation rules.

Phase 2: Suggestion UI

1. Build floating suggestion overlay.
2. Display primary suggestion.
3. Add accept and ignore shortcuts.
4. Add correction revert behaviour.
5. Add basic cursor-following behaviour.

Phase 3: Learning

1. Track accepted suggestions.
2. Track ignored suggestions.
3. Track reverted corrections.
4. Implement personal dictionary.
5. Implement word and phrase frequency learning.
6. Improve suggestion ranking using local history.

Phase 4: Settings App

1. Add assistant on/off.
2. Add correction strength.
3. Add app exclusions.
4. Add personal dictionary management.
5. Add learning controls.
6. Add privacy controls.

Phase 5: Local AI Layer

1. Add modular local AI provider interface.
2. Add optional local model backend.
3. Use AI only for non-blocking suggestions.
4. Add cancellation for outdated suggestions.
5. Add fallback when AI is unavailable.

Phase 6: Polish and Reliability

1. Improve overlay animation.
2. Improve app-specific behaviour.
3. Improve sensitive field detection.
4. Add logging without sensitive text.
5. Add crash recovery.
6. Package as installer.
7. Test across common apps.

19. Definition of Done for MVP

The MVP is complete when:

1. The app can run in the background on the target OS.
 2. The app can detect typing in supported apps.
 3. The app can correct common typos, apostrophes, and capitalisation.
 4. The app can show a suggestion box near the typing area.
 5. The user can accept or ignore suggestions using keyboard shortcuts.
 6. The user can revert the last correction.
 7. The app stores user learning data locally.
 8. The app improves suggestions based on accepted and ignored suggestions.
 9. The user can manage a personal dictionary.
 10. The user can disable the assistant per app.
 11. The app avoids sensitive fields where detectable.
 12. The app remains usable without internet access.
 13. The app remains usable without a local AI model.
 14. The app has a minimal settings UI.
 15. The app can be packaged and launched like a normal desktop application.
-

20. Final Product Positioning

Keyboard Assistant is a minimal, local-first desktop typing assistant that helps users type more accurately and naturally across their computer. It corrects obvious mistakes, suggests better words, predicts likely next words, and learns from the user over time while keeping all personal typing data on the device.

The product should feel intelligent but not intrusive. It should be useful on first install, improve with use, and stay out of the way when assistance is not needed.